

總覽：AI 的三種效益站在同一塊地基 CI/CD 上，而這塊地基現在是空的

Agentic AI 帶來的三種效益（對開發時間與風險）

後面十頁是這一頁的展開



現狀：這塊地基是空的。查判靠人、環境靠記憶、main 隨時會壞；上面三種效益各變成——

x 查判靠人

每圈等人；agent 再多也等於一個

x 環境各不同

PPA 比不了；N 次都要人跑

x 交接靠文件加記憶

下游等人；agent 接不了

N 個 agent = x1

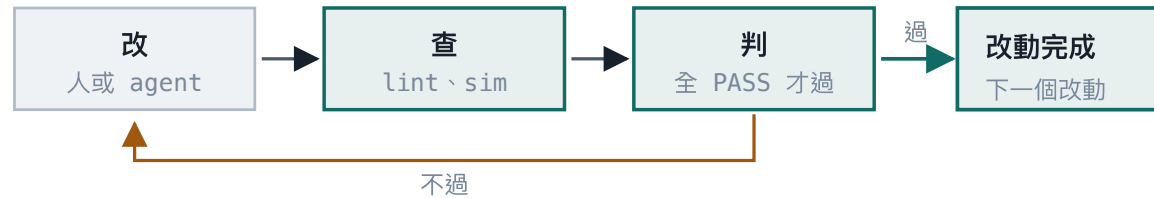
AI 加速的只有「改」；查和判沒有機器做，三種效益一個都拿不到，N 個 agent 等於一個。

把這塊地基補起來，是拿到 Agentic AI 效益的前提；它和買哪一家的 AI 無關。

內圈 (inner loop)：一個改動是改、查、判轉到過為止，查和判交給機器

第一層 內圈

內圈 (inner loop)：一個改動 (加功能、修 bug) = 改、查、判，轉到過為止



CI/CD 做的事：查和判由機器做，每一圈一樣

一個改動的例子 (示意)

改動	給 DMA 加 burst 模式
改	改 RTL，加一個 testcase
查	跑 lint、compile、DMA 的 sim 子集
判	全部 PASS 才過；有一個 FAIL 就回去改

用語：軟體業把 commit 前叫 inner loop (本機的改、lint、unit test)，commit 後的整合與 CI/CD 叫 outer loop。

這份文件的內圈 (inner loop) 指「一個改動的迴圈」，外圈 (outer loop) 指第 7 頁的「探索」：比 N 個方案的 PPA。

CI/CD 把查和判交給機器：每一圈一樣的環境、一樣的標準、不用人在旁邊。

人或 agent 只負責改；內圈什麼時候轉、轉幾圈，都和人的空檔無關。

內圈在 IC：一圈有快有慢，CI/CD 照快慢排成幾道檢查，每道機器判

一圈要多久

長條是示意，不代表任何實際時間

CI/CD 怎麼排

第一層 內圈

lint、compile

快



每次改完就跑

人或 agent 等得起

sanity sim (子集)

中

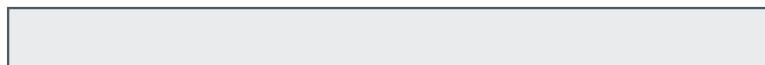


每包 **submit** 都跑

排進 queue，結果回來機器判

完整 regression

慢

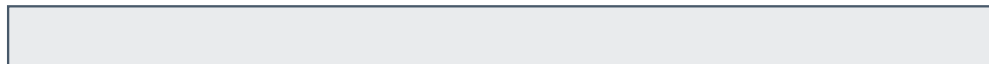


每晚跑一次

早上看結果，附 CL 號

synthesis、STA

很慢



每個里程碑、每個候選各跑一次

跑完自動抓 PPA

快的每圈都跑、慢的定時跑；哪一道都由機器判，結果放在大家看得到的地方。

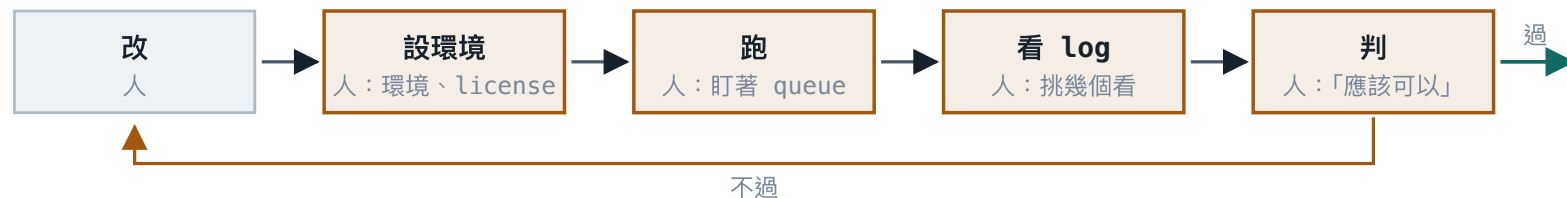
CI/CD 不會讓 sim 或 synthesis 變快；它做的是讓每一道不用人顧、結果能信，慢的那幾道在背景轉。

agent 能自己轉的是快的那幾道；慢的那幾道由它排進去、等結果。

現狀的內圈：每圈靠人設環境、人跑、人看 log，轉得慢、判法還不一

同一個改動，沒有 CI/CD 的時候：內圈長這樣

第一層 內圈



每一圈都卡在人身上；agent 改完也一樣，要等人設環境、人跑、人判。

每一圈的代價

- ✗ 一圈的時間 = 那個人的空檔
- ✗ 兩圈的環境不保證一樣，比不出好壞
- ✗ 判的標準每個人不同
- ✗ 結果對不回是哪一圈改的

圖 1

agent 的處境

圖 14

- 改得很快，然後停下來等人

圖 15

- 一步一步等，速度等於人的速度

圖 4

- 平行幾個 agent 也一樣，排同一條人龍

括號裡的圖號指《把版控當備份的團隊》

沒有 CI/CD，一圈裡四個步驟靠人：設環境、跑、看、判；迴圈轉多快由人的空檔決定。

兩圈之間環境和標準不一樣，連「這一圈比上一圈好」都說不準。

中圈：改動機器查過就進 main，問題早且小；waterfall 把整合留到最後

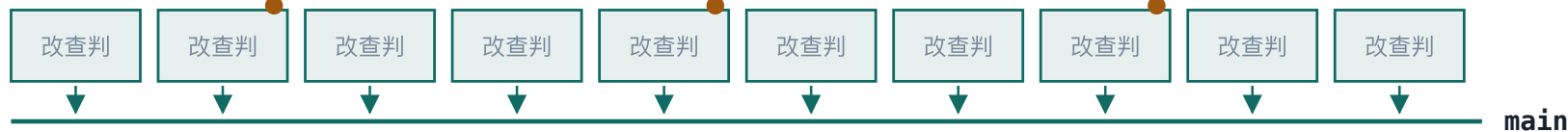
waterfall：每個階段做完才交下去，整合與驗證留到最後

第二層 中圈：一個專案



問題到 DV 和大整合才浮現：interface 對不上、timing 差很多；越晚改越貴（業界叫 shift left：test early and test often）
能交付的狀態：只在最後才有一個；進度 = 文件寫完的比例

中圈（迭代）：每個小改動過了內圈、機器查過就 submit 進 main，main 隨時是能交付的狀態



問題小、早、就在那個改動裡（紅點）；DV 與 synthesis 那幾道檢查跟著每個改動跑（第 3 頁）
能交付的狀態：每個改動進 main 之後都有一個；進度 = 通過檢查的功能數

時間上

- 問題早發現，改的代價小
- 沒有最後的大整合，時程的變異小
- 下游用不著等全部做完才開始（下一頁）

風險上

- 未知早曝光，留到最後的少
- 隨時有 known-good 可退回（《把版控當備份的團隊》圖 16）
- 進度用通過的檢查量，用不著靠文件

迭代把整合和驗證攤到每個改動，問題在小的時候就被看見；waterfall 把它們留到最後整合才全出。

迭代的是改動怎麼整合，tape-out 本身還是一次性的。

中圈的交接 (CD)：改動機器查過就自動交到下一棒，下游人或 agent 不必等人

交接：上游交給下游（架構 → RTL → DV → PD），或同一件工作，白天的人交給晚上的 agent

第二層 中圈：一個專案



agent 接得了的那一段（務實）

- 打包、跑 check、修 lint、讓測試過、附 manifest
- 人不在的時候讓內圈繼續轉

還是人的那一段

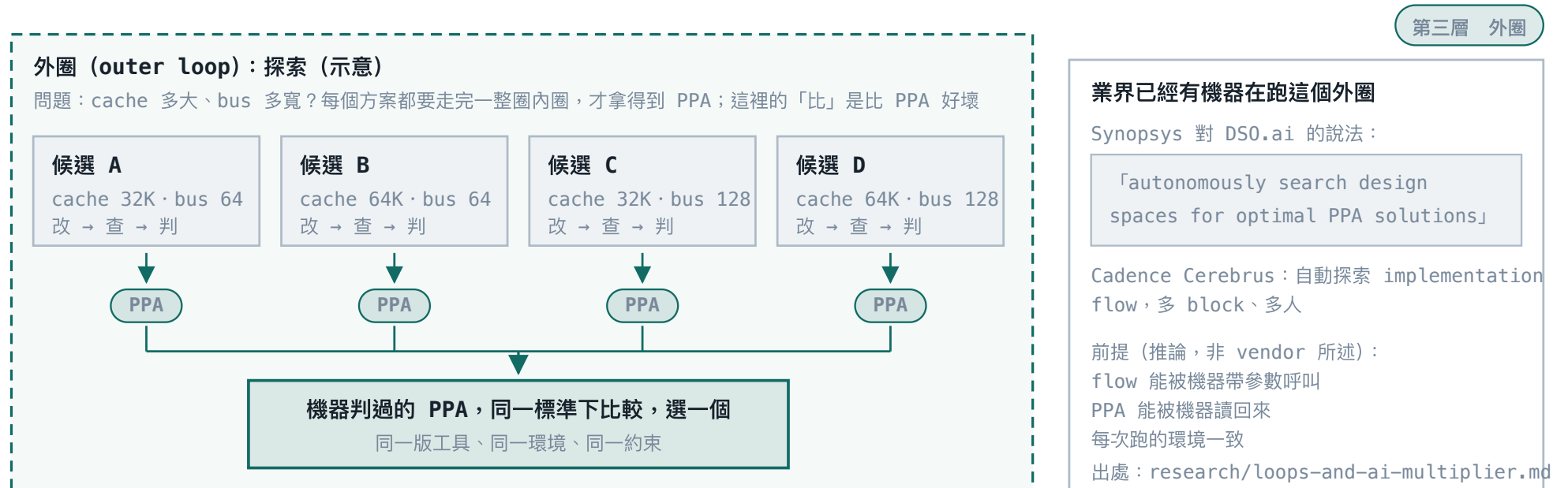
- 設計對不對、取捨怎麼選（人看）
- 下游接下來做什麼

跨團隊交 spec 文件時，waterfall 是唯一能協調的方式；交出去的東西換成跑得起來的，迭代才過得了交界。

迭代的每次交接小而查過，下游不用等；交得給人，也交得給 agent。

交出去的是文件加記憶，agent 接不了；交出去的是版控裡跑得起來的狀態，agent 才接得了。

外圈 (outer loop) : N 個方案比 PPA，每個先各自轉完內圈、機器判過才比



要比 N 個方案，每個方案都要走完一整圈內圈，拿到同一標準下的 PPA。

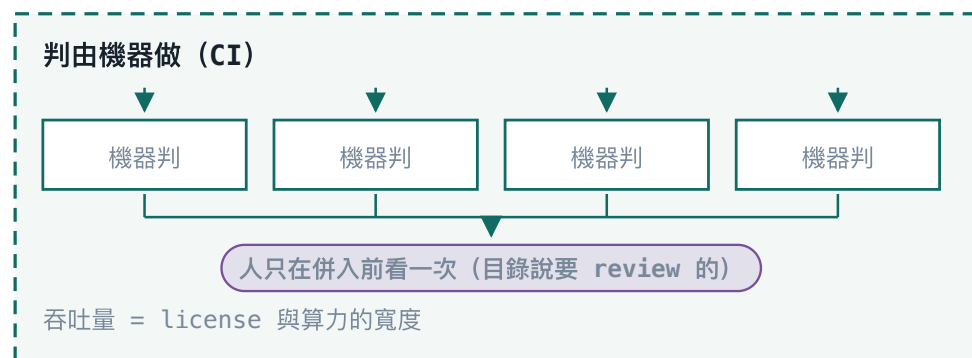
內圈轉不起來，外圈就沒有東西可以比；買了 DSO.ai 這類工具也接不上。

Agentic AI：agent 在三層都加速「改」，瓶頸變成誰來查、誰來判

agent 加速的是「改」：自己轉內圈、接交接、平行跑 N 個方案（產生候選、改 RTL、寫 testbench）



然後每一個都要被「判」



管理平行 coding agent 的工具 Superset 在部落格寫的：「every agent needs a human to review its code... it's the humans that don't scale」
另一位實作者的結論：產生與驗證可以平行，語意上的接受要串行。出處見 [research/loops-and-ai-multiplier.md](#)

agent 讓「改」變得很便宜，平行灑 N 個之後，瓶頸整個移到「判」。

判由人做，N 個 agent 排成一條人龍；判由機器做，人只在併入前看一次。

上限：同時轉幾個方案看 license 與算力；交給機器排，license 才排得滿

平行的寬度

由 license 與算力決定

- N 個 agent 要 N 份 license
- agent 再多也排隊
- 寬度是 CAD 與預算的事

一圈的底線

heavy flow 一圈仍然慢

- regression、synthesis 一圈還是那麼久
- AI 省的是等人的時間
- 省不了工具跑的時間

比較的前提

同一版工具、環境、約束

- 少一樣，PPA 的差異
- 就分不出是候選的差
- 還是環境的差（圖 14）

AI 的加成 = 不用人顧的圈數 × 可信的比較

把 AI 的效益講成「sim 變快」是講錯了；它加的是圈數和比較，而且三個上限都還在

CI/CD 直接解的是第三個（比較的前提）；前兩個是預算的事，但 license 要不用等人有空就排得滿，也得先有 CI/CD。

平行有上限，一圈有底線，比較有前提；三個都是工程問題，講清楚效益才算得出來。

CI/CD 解的是比較的前提，也讓 license 和算力不用等人有空就排得滿。

結論：沒有 CI/CD，每圈、每次交接、每個方案都要等人查判，N 個 agent 等於一個



同樣 N 個 agent、同樣的 license，一段時間內轉完的圈數（長條是示意）



《把版控當備份的團隊》的症狀，各打斷鏈上哪一環

圖 2、3 環境散在 depot 外 → 每圈要人設環境

圖 14 兩台機器結果不同 → 圈與圈不可比

圖 15 狀態靠人填 Excel → 判靠人

圖 5 壞了很久才發現 → 圈沒有閉合

圖 8 main 隨時會壞 → agent sync 到壞的

沒有 CI/CD，每圈、每次交接、每個方案都要等人查判；AI 加速的只有「改」，N 個 agent 的效果等於一個。

《把版控當備份的團隊》裡的每個症狀，都在這條鏈上打斷一環。

對應：三層各要的條件，對到《把版控當備份的團隊》的六個原則

哪一層	要的條件	撐住它的原則（出自《把版控當備份的團隊》）	少了會怎樣
內圈	一圈一件事	Small batches	判出來的差異才歸得到某個改動
內圈	查和判由機器做	Continuous Integration	少了：每圈要人
中圈	人判放在進 main 前一次做完，每個目錄講好要不要	Self-documenting (review 規矩)	少了：人判散在每一圈，或沒人知道要不要看
中圈	交接時 agent 不用人帶就看得懂	Self-documenting	少了：每個 workspace 要人寫說明
中圈	過了 check 就自動交到下一棒，下游拿了就能跑	Continuous Delivery	少了：交接靠 email 與人解釋
外圈	每個方案的環境一樣，agent 進得去就能跑	Single Source of Truth	少了：方案不可比，agent 要人設環境
外圈	每個結果連得回方案與環境	Traceability	少了：比較表是假的

三層各要的條件，都是前面幾頁講過的事；撐住它們的六個原則（加一條 review 規矩），另一份文件已經收斂出來。
這就是 CI/CD 對這個團隊的意義，也是 N 個 agent 要有 N 倍效果的前提。